



Instituto Politécnico Nacional

Escuela Superior de Computo

Unidad de aprendizaje:

Paradigmas de programación

Práctica integradora 1er Parcial:

SEOV-Pro

Sistema Experto de Orientación Vocacional

Integrantes:

Lopez Teran Monserrat

García Ramírez Saúl Maximiliano

Grupo:3CV3

Contenido

Introducción 1

Planteamiento del problema 2

Objetivo general 3

Objetivos específicos 3

Justificación del uso de Prolog 5

Descripción de la base de conocimientos 6

Explicación de los hechos y reglas más importantes 10

Interpretación de los resultados 12

Diagrama de arquitectura del sistema 12

Descripción del funcionamiento de la interfaz gráfica 14

Tecnologías utilizadas 17

Procedimiento de integración y despliegue 20

Problemas encontrados y soluciones implementadas 23

Conclusiones 25

Referencias bibliográficas 27

Declaración del uso de herramientas de inteligencia artificial 28

Introducción

La orientación vocacional representa un proceso importante en la formación académica de los estudiantes, ya que permite identificar áreas profesionales relacionadas con sus intereses, habilidades y preferencias. En este contexto, los sistemas expertos constituyen una alternativa tecnológica para apoyar la toma de decisiones mediante el uso de conocimientos estructurados y reglas de inferencia. El presente proyecto, denominado SEOV-Pro, consiste en el desarrollo de un sistema experto de orientación vocacional implementado con SWI-Prolog, que permite analizar las características proporcionadas por un estudiante y generar recomendaciones de carreras profesionales con distintos niveles de compatibilidad.

¿Qué es un sistema experto y cómo utiliza una base de conocimientos?

Un sistema experto es un programa informático perteneciente al campo de la inteligencia artificial que emula el razonamiento y la capacidad de resolución de problemas de un especialista humano en un dominio específico. Arquitecturalmente, estos sistemas se componen principalmente de una base de conocimientos (que almacena los hechos y reglas del dominio) y un motor de inferencia (el mecanismo lógico que procesa dichos datos). En lugar de seguir un flujo de control procedimental tradicional, el sistema experto aplica reglas lógicas sobre los datos de entrada para deducir conclusiones de manera automática.

La programación lógica se fundamenta en la lógica matemática formal. a diferencia de los lenguajes imperativos, donde se detalla el paso a paso de cómo la computadora debe resolver un problema, la programación lógica es de naturaleza declarativa: el programador define *qué* es verdad mediante hechos y reglas, y el motor de inferencia calcula *cómo* llegar a la solución. En este marco, SWI-Prolog destaca como una de las herramientas más potentes, permitiendo modelar relaciones complejas de manera concisa y procesar consultas de usuarios con gran eficiencia.

La elección de una carrera profesional no responde a una simple fórmula matemática, sino a una combinación de factores cualitativos: áreas de conocimiento preferidas, habilidades o competencias específicas y el entorno laboral deseado. Este dominio es idóneo para ser modelado mediante reglas de producción, ya que las condiciones y criterios de afinidad de un experto vocacional pueden traducirse directamente en sentencias lógicas (por ejemplo, determinar si un perfil presenta una compatibilidad alta, moderada o si requiere rutas de exploración interdisciplinaria).

Planteamiento del problema

La elección de una carrera profesional constituye una de las decisiones más importantes durante la formación académica de los estudiantes, debido a que influye directamente en su desarrollo educativo, sus oportunidades laborales y sus expectativas personales. Sin embargo, este proceso puede representar una dificultad cuando no se cuenta con información suficiente acerca de las diferentes áreas profesionales o cuando resulta complicado identificar la relación entre los intereses, habilidades y preferencias individuales con las opciones académicas disponibles.

Uno de los principales problemas que enfrentan los estudiantes durante el proceso de orientación vocacional es la falta de claridad respecto a sus propias aptitudes y su posible aplicación en determinadas profesiones. En muchas ocasiones, la elección de una carrera puede estar influenciada por factores externos, como las tendencias sociales o la percepción de las oportunidades económicas, sin considerar suficientemente las características personales del estudiante. Esta situación puede generar incertidumbre y dificultar la identificación de alternativas profesionales acordes con su perfil.

Desde el punto de vista informático, esta problemática representa una oportunidad para aplicar los principios de la programación lógica mediante la construcción de un sistema experto. Este tipo de sistemas permite representar conocimientos de un dominio específico mediante hechos y reglas, los cuales pueden evaluarse a través de un motor de inferencia para obtener conclusiones a partir de los datos proporcionados por el usuario.

En respuesta a esta necesidad, se propone el desarrollo de **SEOV-Pro (Sistema Experto de Orientación Vocacional)**, una aplicación web diseñada para analizar tres características principales del perfil de un estudiante: el área de interés profesional, la habilidad o aptitud destacada y el ambiente de trabajo preferido. A partir de estos elementos, el sistema consulta una base de conocimientos desarrollada en SWI-Prolog y aplica reglas de inferencia para identificar carreras que presentan diferentes niveles de compatibilidad con las preferencias seleccionadas.

El problema central que aborda el proyecto consiste en determinar cómo representar y relacionar lógicamente las características de distintas carreras profesionales con las preferencias de un estudiante, de manera que sea posible generar recomendaciones mediante un proceso automatizado, estructurado y comprensible. Para resolverlo, es necesario definir los hechos que describen las carreras, establecer las reglas que determinan sus coincidencias y desarrollar un mecanismo de comunicación entre el motor lógico y una interfaz gráfica accesible para el usuario. El sistema se plantea como una herramienta de apoyo para la exploración de alternativas profesionales y como una demostración práctica de la aplicación de la programación lógica en la resolución de problemas.

Objetivo general

Desarrollar e implementar un sistema experto de orientación vocacional mediante el lenguaje de programación lógica SWI-Prolog, capaz de analizar las áreas de interés, habilidades y preferencias de ambiente laboral de los estudiantes, con la finalidad de generar recomendaciones de carreras profesionales a partir de una base de conocimientos estructurada y reglas de inferencia lógica, integrando una interfaz web interactiva que facilite la consulta, visualización e interpretación de los resultados obtenidos.

Objetivos específicos

1. Diseñar una base de conocimientos: que represente diferentes carreras profesionales mediante hechos y relaciones lógicas, considerando características como el área de conocimiento, las habilidades requeridas y los ambientes laborales asociados a cada profesión.

2. Implementar reglas de inferencia en SWI-Prolog: que permitan evaluar las coincidencias entre las características proporcionadas por el estudiante y la información registrada en la base de conocimientos, estableciendo diferentes niveles de compatibilidad para las recomendaciones generadas.

3. Aplicar los principios de programación lógica: como la unificación, la evaluación de predicados y el retroceso (*backtracking*), para identificar las carreras profesionales que satisfacen total o parcialmente los criterios seleccionados por el usuario.

4. Desarrollar una interfaz gráfica web interactiva**:** utilizando HTML, CSS y JavaScript, que permita a los estudiantes seleccionar sus áreas de interés, habilidades destacadas y preferencias de ambiente laboral mediante un formulario intuitivo y accesible.

5. Establecer la comunicación entre el frontend y el backend**:** mediante peticiones HTTP y respuestas en formato JSON, permitiendo que las consultas realizadas desde la interfaz web sean procesadas por el motor de inferencia desarrollado en SWI-Prolog.

6. Presentar recomendaciones profesionales comprensibles: mostrando el nombre de las carreras identificadas, su nivel de compatibilidad y una explicación general de los criterios utilizados durante el proceso de inferencia lógica.

7. Implementar una arquitectura cliente-servidor: que permita ejecutar de manera independiente la interfaz gráfica y el motor de inferencia, utilizando Docker para contenerizar el backend y servicios de alojamiento web para facilitar el acceso al sistema.

8. Publicar el sistema en plataformas de despliegue web: utilizando Render para alojar el servidor SWI-Prolog y Vercel para la interfaz gráfica, con el propósito de permitir el acceso a la aplicación a través de una dirección pública.
9. Utilizar Git y GitHub para gestionar el código fuente: organizar los archivos del proyecto, documentar su funcionamiento y mantener un historial de modificaciones mediante commits descriptivos.
10. Realizar pruebas funcionales del sistema: utilizando diferentes combinaciones de áreas, habilidades y ambientes laborales para comprobar que las reglas de inferencia generen los resultados esperados y que la comunicación entre la interfaz web y SWI-Prolog funcione correctamente.

Justificación del uso de Prolog

El desarrollo de SEOV-Pro (Sistema Experto de Orientación Vocacional) requiere una tecnología que permita representar conocimientos, establecer relaciones entre diferentes elementos y obtener conclusiones mediante la evaluación de condiciones lógicas. Por esta razón, se seleccionó el lenguaje de programación Prolog, específicamente su implementación SWI-Prolog, debido a que proporciona herramientas adecuadas para construir sistemas basados en conocimientos y reglas de inferencia.

Prolog, cuyo nombre proviene de *Programming in Logic* (Programación en Lógica), es un lenguaje de programación declarativo fundamentado en principios de la lógica formal. A diferencia de los lenguajes de programación imperativos, en los cuales se establece una secuencia de instrucciones para resolver un problema, Prolog permite describir hechos y relaciones que representan el conocimiento de un dominio específico. A partir de esta información, el intérprete puede evaluar consultas y determinar las soluciones que satisfacen las condiciones establecidas.

Una de las principales razones para utilizar Prolog en SEOV-Pro es su capacidad para representar información mediante hechos. Estos constituyen afirmaciones que el sistema considera verdaderas dentro de su base de conocimientos. En el proyecto, los hechos permiten registrar las carreras profesionales disponibles, sus áreas de conocimiento, las habilidades relacionadas con cada profesión y los ambientes laborales asociados. Esta organización facilita la representación estructurada de la información vocacional.

Otra característica fundamental es la utilización de reglas de inferencia, las cuales permiten establecer conclusiones a partir de condiciones previamente definidas. En SEOV-Pro, las reglas analizan tres criterios proporcionados por el estudiante: el área de interés, la habilidad destacada y el entorno laboral preferido. Dependiendo de las coincidencias identificadas, el sistema determina diferentes niveles de compatibilidad, como alta, moderada o exploratoria. De esta manera, las recomendaciones se obtienen mediante la evaluación de relaciones lógicas y no a través de respuestas seleccionadas arbitrariamente.

Asimismo, Prolog utiliza un mecanismo denominado unificación, que consiste en establecer correspondencias entre términos y variables durante la evaluación de una consulta. En SEOV-Pro, este mecanismo permite relacionar los valores seleccionados por el estudiante con los hechos registrados en la base de conocimientos. Por ejemplo, cuando un usuario selecciona el área de ciencias exactas y la habilidad de programación, el motor lógico busca las carreras que satisfacen dichas características.

Otro aspecto relevante es el mecanismo de retroceso o backtracking, mediante el cual Prolog puede explorar diferentes alternativas cuando existen varias soluciones posibles. Esta característica resulta útil en el sistema, debido a que una misma área profesional puede contener distintas carreras y una habilidad puede estar relacionada con más de una opción académica. En lugar de limitarse a encontrar una única coincidencia, Prolog puede continuar evaluando las relaciones disponibles para identificar diferentes recomendaciones.

La elección de **SWI-Prolog** también se justifica por las bibliotecas que proporciona para desarrollar aplicaciones con comunicación HTTP. Mediante estas herramientas fue posible implementar un servidor que recibe las preferencias seleccionadas desde la interfaz gráfica, ejecuta las reglas de inferencia y devuelve los resultados en un formato estructurado. Esto permitió integrar la programación lógica con tecnologías web como HTML, CSS y JavaScript, estableciendo una separación entre la presentación visual y el procesamiento del conocimiento.

Descripción de la base de conocimientos

-Estructura general de la base de conocimientos

La base de conocimientos constituye uno de los componentes fundamentales del Sistema Experto de Orientación Vocacional (SEOV-Pro), debido a que almacena la información necesaria para que el motor de inferencia pueda analizar las preferencias proporcionadas por los estudiantes y generar recomendaciones de carreras profesionales.

En este proyecto, la base de conocimientos fue desarrollada utilizando el lenguaje SWI-Prolog y se encuentra definida en el archivo servidor_web.pl. Su estructura está conformada por hechos y reglas lógicas que permiten representar diferentes carreras profesionales y establecer relaciones entre sus características principales.

Los hechos describen información considerada verdadera dentro del dominio del sistema, mientras que las reglas permiten establecer condiciones para determinar la compatibilidad entre las preferencias del usuario y las carreras registradas.

La base de conocimientos de SEOV-Pro considera tres criterios principales para realizar las recomendaciones:

1. **Área de conocimiento:** representa el campo académico al que pertenece una carrera profesional, como ciencias exactas, ciencias de la salud, ciencias sociales, diseño o áreas económico-administrativas.
2. **Habilidades relacionadas:** representan capacidades o aptitudes asociadas con determinadas profesiones, como programación, pensamiento lógico, biología, creatividad, liderazgo o comunicación.
3. **Entorno laboral:** representa el ambiente de trabajo asociado con una carrera, como hospitales, oficinas, tribunales, actividades de campo o modalidades de trabajo remoto.

La combinación de estos elementos permite que el sistema identifique diferentes niveles de coincidencia y genere recomendaciones mediante la aplicación de reglas de inferencia lógica.

-Carreras profesionales registradas

La versión desarrollada de SEOV-Pro contiene ocho carreras profesionales, organizadas según su área de conocimiento.

Identificador	Carrera profesional	Área de conocimiento
Sistemas	Ingeniería en Sistemas Computacionales	Ciencias exactas
Medicina	Medicina Humana	Ciencias de la salud
Derecho	Licenciatura en Derecho	Ciencias sociales
Administración	Administración de Empresas	Económico-administrativas
Psicología	Licenciatura en Psicología	Ciencias de la salud
Arquitectura	Arquitectura	Diseño
Mecánica	Ingeniería Mecánica	Ciencias exactas
Diseño_digital	Diseño y Animación Digital	Diseño

Cada carrera cuenta con un identificador único que permite relacionarla con sus habilidades y entornos laborales mediante los diferentes predicados definidos en Prolog. Por ejemplo, el identificador sistemas representa la carrera de Ingeniería en Sistemas Computacionales y permite asociarla con habilidades como programación y pensamiento lógico.

-Predicados utilizados en la base de conocimientos

En Prolog, un predicado permite representar hechos, relaciones o reglas mediante un nombre y una cantidad determinada de argumentos. Esta cantidad se conoce como aridad y se representa utilizando la notación nombre/aridad.

En SEOV-Pro se utilizan los siguientes predicados principales:

Predicado	Descripción
carrera/3	Registra el identificador, nombre y área de conocimiento de una carrera.
requiere_habilidad/2	Relaciona una carrera con una habilidad.
requiere_entorno/2	Relaciona una carrera con un ambiente laboral.
evaluar_compatibilidad/5	Evalúa las preferencias del usuario y determina el nivel de compatibilidad de una carrera.

El predicado carrera/3 recibe tres argumentos: identificador, nombre de la carrera y área de conocimiento.

El predicado requiere_habilidad/2 establece una relación entre el identificador de una carrera y una habilidad determinada.

Por su parte, requiere_entorno/2 relaciona una carrera profesional con el ambiente laboral asociado.

Finalmente, evaluar_compatibilidad/5 constituye el principal predicado de inferencia del sistema, debido a que utiliza la información almacenada en los hechos para determinar qué carreras cumplen total o parcialmente con las características seleccionadas por el estudiante.

-Representación de las habilidades profesionales

Las habilidades profesionales permiten identificar capacidades relacionadas con el desempeño de determinadas carreras. En SEOV-Pro, estas asociaciones se representan mediante hechos del predicado requiere_habilidad/2.

Una misma carrera puede tener varias habilidades registradas, lo que permite evaluar diferentes perfiles de estudiantes. Por ejemplo, Ingeniería en Sistemas Computacionales se encuentra relacionada con programación y pensamiento lógico, mientras que Medicina Humana está asociada con biología y empatía.

Esta representación permite que el motor de inferencia identifique carreras que coinciden con las habilidades seleccionadas por el usuario, sin necesidad de almacenar manualmente todas las combinaciones posibles de respuestas.

-Representación de los entornos laborales

El entorno laboral constituye el tercer criterio utilizado por SEOV-Pro para generar recomendaciones. Este elemento representa las condiciones generales del ambiente en el que se desarrolla una profesión.

Las relaciones entre carreras y entornos se registran mediante el predicado requiere_entorno/2.

Por ejemplo, Ingeniería en Sistemas Computacionales se relaciona con un entorno remoto u oficina, Medicina Humana con hospitales e Ingeniería Mecánica con actividades de campo y oficina.

La incorporación de este criterio permite realizar una evaluación más específica de las preferencias del estudiante, considerando no solamente sus intereses académicos y habilidades, sino también el tipo de ambiente laboral en el que preferiría desempeñarse.

Ejemplo 1. Registro de carreras

```
carrera(sistemas, 'Ingenieria en Sistemas Computacionales', exactas).
carrera(medicina, 'Medicina Humana', salud).
carrera(derecho, 'Licenciatura en Derecho', sociales).
carrera(administracion, 'Administracion de Empresas', economico_administrativas).
```

El predicado carrera/3 registra tres elementos: identificador, nombre y área de conocimiento.

Ejemplo 2. Asociación de habilidades

```
requiere_habilidad(sistemas, programacion).
requiere_habilidad(sistemas, pensamiento_logico).
requiere_habilidad(medicina, biologia).
requiere_habilidad(medicina, empatia).
```

Estos hechos establecen relaciones entre las carreras y las habilidades correspondientes.

Ejemplo 3. Asociación de entornos laborales

```
requiere_entorno(sistemas, remoto_u_oficina).
requiere_entorno(medicina, hospital).
requiere_entorno(derecho, oficina_o_tribunal).
requiere_entorno(mecanica, campo_y_oficina).
```

Cada hecho establece una relación entre una carrera y un entorno laboral.

Ejemplo de consulta a la base de conocimientos

Para demostrar cómo se utiliza la información:

```
?- carrera(Id, Nombre, exactas).
```

La consulta pregunta qué carreras pertenecen al área de ciencias exactas.

Prolog puede devolver las siguientes soluciones mediante backtracking:

```
Id = sistemas,
Nombre = 'Ingenieria en Sistemas Computacionales' ;

Id = mecanica,
Nombre = 'Ingenieria Mecanica'.
```

Este ejemplo demuestra que Prolog puede recuperar diferentes soluciones utilizando los hechos registrados en la base de conocimientos.

Explicación de los hechos y reglas más importantes

Fundamentos de la inferencia lógica en SEOV-Pro

El funcionamiento del Sistema Experto de Orientación Vocacional (SEOV-Pro) se fundamenta en la utilización de hechos y reglas de inferencia desarrollados mediante el lenguaje SWI-Prolog. Estos elementos permiten representar información sobre distintas carreras profesionales y establecer condiciones lógicas para determinar cuáles presentan mayor coincidencia con las características seleccionadas por el estudiante.

Los hechos constituyen afirmaciones almacenadas en la base de conocimientos, mientras que las reglas permiten obtener conclusiones a partir de las relaciones existentes entre dichos hechos. Mediante este mecanismo, el sistema puede analizar diferentes combinaciones de áreas de interés, habilidades y ambientes laborales, generando recomendaciones sin necesidad de programar una respuesta específica para cada combinación posible.

El principal predicado encargado de realizar este proceso es evaluar_compatibilidad/5, el cual recibe cinco argumentos: área de conocimiento, habilidad, entorno laboral, identificador de la carrera y nivel de compatibilidad.

Los primeros tres argumentos representan las preferencias seleccionadas por el estudiante, mientras que los dos últimos permiten identificar la carrera evaluada y el nivel de coincidencia obtenido.

Regla de compatibilidad alta

La primera regla establece que una carrera presenta compatibilidad alta cuando coincide con los tres criterios seleccionados por el estudiante: área de conocimiento, habilidad y entorno laboral.

Esta regla utiliza una conjunción de condiciones lógicas, por lo que todas deben cumplirse para que la recomendación sea clasificada como alta. El nivel denominado *Alta (100%)* representa una coincidencia completa con los tres criterios considerados por el sistema.

Regla de compatibilidad moderada

La segunda regla permite identificar carreras que coinciden con el área de conocimiento y la habilidad seleccionadas, pero cuyo entorno laboral registrado es diferente del preferido por el estudiante.

En este caso, el sistema considera que existe una relación significativa entre el perfil del usuario y la carrera, aunque no se satisfacen todos los criterios establecidos.

Para implementar esta condición se utiliza el operador de negación por fallo \+, el cual permite comprobar que una determinada consulta no puede demostrarse utilizando la información disponible en la base de conocimientos.

De esta manera, una carrera puede continuar siendo recomendada aun cuando el entorno laboral no coincida con la preferencia indicada.

Regla de exploración por área de conocimiento

La tercera regla identifica carreras que pertenecen al área de conocimiento seleccionada por el estudiante, pero que no presentan coincidencia con la habilidad indicada.

Este nivel se denomina *Exploración (Solo Área)* y permite mostrar alternativas profesionales relacionadas con el campo académico de interés del usuario, aunque no coincidan con todas sus características.La finalidad de esta regla es ampliar las opciones presentadas por el sistema, evitando que las recomendaciones se limiten exclusivamente a coincidencias completas.

Regla de exploración por habilidad

La cuarta regla contempla la posibilidad de identificar carreras relacionadas con la habilidad seleccionada por el estudiante, aunque pertenezcan a un área de conocimiento diferente de la indicada.

Este nivel se denomina *Exploración (Solo Habilidad)* y busca ofrecer alternativas que permitan al usuario considerar profesiones fuera de su área de interés inicial.

Mecanismos de programación lógica utilizados

Durante la ejecución de las reglas, SWI-Prolog utiliza diferentes mecanismos para evaluar las condiciones establecidas.

La unificación permite relacionar los argumentos de una consulta con los hechos almacenados en la base de conocimientos. Por ejemplo, cuando el usuario selecciona el área de ciencias exactas, Prolog busca los hechos correspondientes a las carreras registradas en dicha área.

La conjunción lógica permite establecer que varias condiciones deben cumplirse simultáneamente para que una regla tenga éxito. En Prolog, esta operación se representa mediante una coma.

La negación por fallo permite comprobar que una condición no puede demostrarse con la información disponible. Este mecanismo se representa mediante el operador \+ y se utiliza para diferenciar determinados niveles de compatibilidad.

Por último, el backtracking permite explorar diferentes soluciones cuando una consulta puede satisfacerse mediante más de un hecho o regla. Gracias a este mecanismo, el sistema puede identificar varias carreras relacionadas con las características seleccionadas.

Interpretación de los resultados

Las recomendaciones obtenidas por el motor de inferencia se organizan según los niveles de coincidencia establecidos en las reglas.

Nivel de compatibilidad	Criterios considerados
Alta (100%)	Coincidencia de área, habilidad y entorno.
Moderada (Área + Habilidad)	Coincidencia de área y habilidad, sin coincidencia del entorno.
Exploración (Solo Área)	Coincidencia del área, sin coincidencia de la habilidad.
Exploración (Solo Habilidad)	Coincidencia de habilidad con una carrera de otra área, según la lógica implementada.

Diagrama de arquitectura del sistema

-Descripción general de la arquitectura

El Sistema Experto de Orientación Vocacional (SEOV-Pro) fue desarrollado utilizando una arquitectura cliente-servidor, en la cual los componentes encargados de la presentación de información y del procesamiento lógico se encuentran separados.

Esta organización permite que la interfaz gráfica y el motor de inferencia funcionen de manera independiente, comunicándose mediante peticiones HTTP y respuestas estructuradas en formato JSON.

La arquitectura del sistema está integrada por cuatro elementos principales: el navegador web del usuario, el frontend desarrollado con HTML, CSS y JavaScript, el backend implementado en SWI-Prolog y la base de conocimientos conformada por hechos y reglas de inferencia.

El frontend se encuentra publicado mediante la plataforma Vercel, mientras que el backend está alojado en Render, utilizando Docker para proporcionar el entorno de ejecución necesario para SWI-Prolog.

La base de conocimientos forma parte del código del backend, por lo que no se requiere un sistema gestor de bases de datos independiente. Los hechos y reglas son consultados directamente por el motor de inferencia durante el procesamiento de las solicitudes.

-Funcionamiento de la comunicación

La comunicación entre los componentes comienza cuando el estudiante accede a la aplicación web y selecciona sus preferencias mediante el formulario.

Posteriormente, JavaScript obtiene los valores correspondientes al área de conocimiento, la habilidad y el entorno laboral, y construye una solicitud HTTP dirigida al endpoint /recomendar del backend.

El servidor desarrollado en SWI-Prolog recibe los parámetros y utiliza las reglas de inferencia para identificar las carreras que cumplen con las condiciones establecidas.

Una vez obtenidas las recomendaciones, el backend genera una respuesta en formato JSON que contiene el nombre de las carreras y sus niveles de compatibilidad.

Finalmente, el frontend interpreta la respuesta recibida y presenta los resultados mediante elementos visuales que facilitan su lectura, acompañados de información explicativa para el usuario.

Ventajas de la arquitectura implementada

La separación entre frontend y backend permite organizar el proyecto en componentes independientes, facilitando su mantenimiento y actualización. Además, esta arquitectura permite modificar la presentación visual sin alterar necesariamente las reglas de inferencia, así como ampliar la base de conocimientos sin rediseñar completamente la interfaz. El uso de Docker facilita la ejecución del backend en un entorno controlado, mientras que las plataformas de despliegue permiten acceder a la aplicación a través de internet.

En conclusión, la arquitectura cliente-servidor implementada en SEOV-Pro permite integrar una interfaz web interactiva con un motor de inferencia desarrollado en SWI-Prolog, estableciendo un flujo de comunicación que transforma las preferencias del estudiante en recomendaciones profesionales basadas en reglas lógicas.

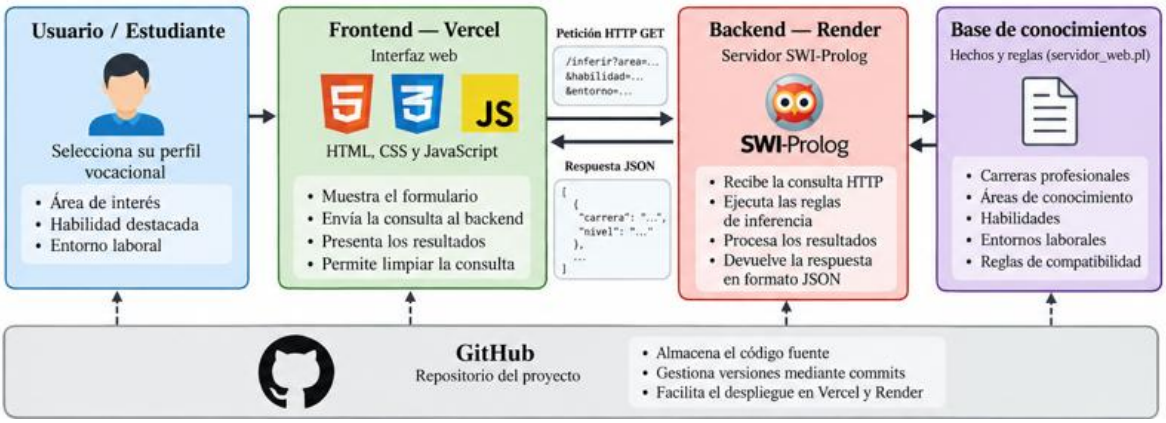


Ilustración 1: Diagrama de arquitectura del sistema SEOV-Pro

Descripción del funcionamiento de la interfaz gráfica

Descripción general de la interfaz

La interfaz gráfica de SEOV-Pro (Sistema Experto de Orientación Vocacional) fue desarrollada utilizando HTML, CSS y JavaScript, con el propósito de proporcionar un entorno visual intuitivo que permita a los estudiantes consultar recomendaciones de carreras profesionales sin necesidad de conocer el funcionamiento interno del lenguaje SWI-Prolog.

El diseño de la aplicación utiliza una presentación de colores oscuros, elementos organizados en secciones y controles de selección que facilitan la interacción con el sistema. La interfaz permite ingresar las características del perfil vocacional, ejecutar una consulta y visualizar las recomendaciones generadas por el motor de inferencia.

La aplicación se encuentra publicada en la plataforma Vercel y puede consultarse mediante la siguiente dirección:

https://seov-pro-frontend.vercel.app

La página principal contiene un formulario denominado *Perfil del Estudiante*, en el cual se recopilan tres características fundamentales: área de interés, habilidad destacada y entorno laboral preferido.

Una vez seleccionados estos elementos, el estudiante puede solicitar el análisis de su perfil mediante el botón de consulta. El sistema procesa la información utilizando el backend desarrollado en SWI-Prolog y presenta las carreras recomendadas junto con sus niveles de compatibilidad.

Formulario del perfil del estudiante

El formulario constituye el principal mecanismo de interacción entre el usuario y SEOV-Pro. Su función consiste en recopilar las preferencias necesarias para que el motor de inferencia realice la evaluación de compatibilidad.

El formulario está integrado por tres campos de selección.

Área de interés: permite seleccionar el campo académico o profesional que resulta más atractivo para el estudiante. Entre las áreas contempladas por la base de conocimientos se encuentran ciencias exactas, ciencias de la salud, ciencias sociales, diseño y económico-administrativas.

Habilidad destacada: permite indicar una capacidad o aptitud que el estudiante considera relevante dentro de su perfil. Las habilidades disponibles están relacionadas con los hechos definidos en la base de conocimientos, como programación, pensamiento lógico, biología, empatía, liderazgo y creatividad, entre otras.

Entorno laboral preferido: permite seleccionar el ambiente de trabajo en el que el estudiante considera que podría desempeñarse de manera más adecuada. Las opciones están relacionadas con los entornos registrados en la base de conocimientos, como trabajo remoto u oficina, hospitales, oficinas o tribunales y actividades de campo.

Estos tres campos proporcionan los parámetros necesarios para ejecutar las reglas de inferencia y determinar los niveles de coincidencia entre las preferencias del estudiante y las carreras profesionales disponibles.

Botón de consulta al motor de inferencia

La interfaz incorpora un botón denominado *Consultar SWI-Prolog*, cuya función es iniciar el proceso de evaluación vocacional. Cuando el estudiante selecciona sus preferencias y presiona este botón, JavaScript obtiene los valores de los campos del formulario y construye una petición HTTP dirigida al backend.

La solicitud contiene los parámetros correspondientes al área, la habilidad y el entorno laboral seleccionados. Posteriormente, el servidor SWI-Prolog procesa estos datos mediante las reglas de inferencia y genera una respuesta estructurada. Este procedimiento permite que el usuario interactúe con el sistema experto mediante una interfaz visual, mientras que el procesamiento lógico se realiza de forma independiente en el servidor.

Presentación de las recomendaciones

Después de procesar la consulta, SEOV-Pro presenta las carreras profesionales identificadas por el motor de inferencia. Cada recomendación contiene el nombre de la carrera y el nivel de compatibilidad correspondiente, determinado mediante la evaluación de los criterios seleccionados.

Los niveles utilizados por el sistema permiten distinguir entre coincidencias completas, coincidencias parciales y alternativas de exploración. Por ejemplo, si el estudiante selecciona el área de ciencias exactas, la habilidad de programación y un entorno de trabajo remoto u oficina, el sistema puede recomendar Ingeniería en Sistemas Computacionales con compatibilidad alta e Ingeniería Mecánica como una alternativa de exploración por área.

Explicación de los resultados

Además de mostrar las carreras recomendadas, la interfaz incorpora un apartado explicativo que ayuda al estudiante a interpretar los niveles de compatibilidad. Este elemento permite relacionar las recomendaciones con los criterios utilizados por el motor de inferencia, proporcionando mayor claridad acerca del funcionamiento del sistema experto.

La explicación de los resultados constituye una característica importante del proyecto, debido a que las recomendaciones no se generan de manera aleatoria, sino a partir de hechos y reglas lógicas previamente establecidos. De esta manera, la interfaz no solamente presenta una lista de carreras, sino que también facilita la comprensión general del proceso de evaluación.

Función del botón Limpiar

La interfaz incluye un botón denominado *Limpiar*, que permite restablecer el formulario y retirar los resultados de la consulta anterior. Esta funcionalidad facilita la realización de nuevas evaluaciones, ya que el estudiante puede modificar sus preferencias y consultar diferentes combinaciones de áreas, habilidades y entornos laborales.

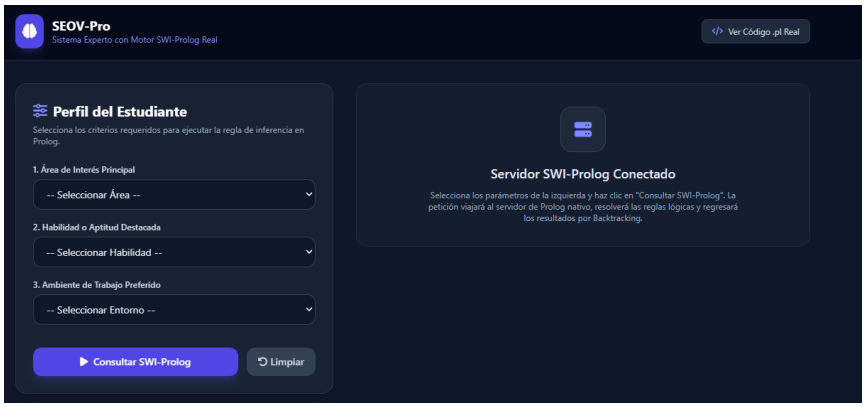


Ilustración 2: Interfaz principal de SEOV-Pro

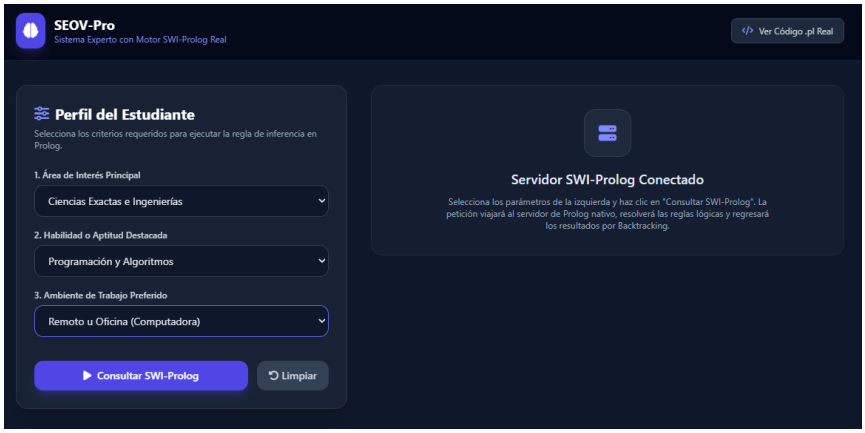


Ilustración 3: Selección del perfil vocacional

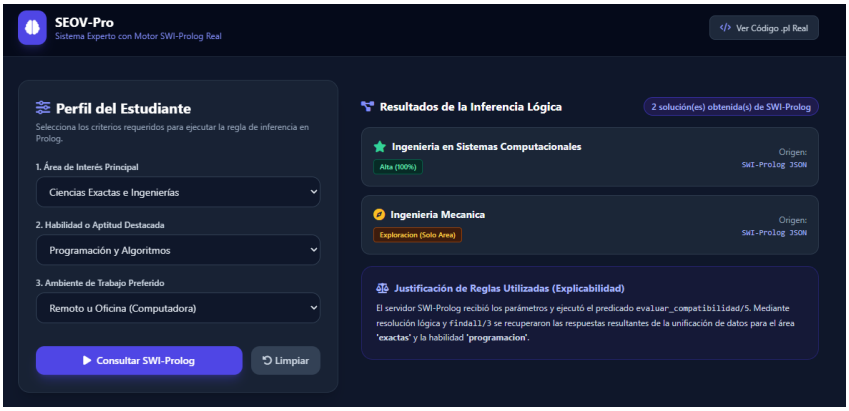


Ilustración 4: Resultados de la evaluación vocacional

Tecnologías utilizadas

Descripción general

El desarrollo e implementación del Sistema Experto de Orientación Vocacional (SEOV-Pro) requirió la integración de diferentes tecnologías orientadas a la programación lógica, el desarrollo web, la comunicación entre aplicaciones, la contenerización y el despliegue de servicios en internet.

La selección de estas herramientas respondió a las necesidades funcionales del proyecto, particularmente a la implementación de un motor de inferencia basado en conocimientos, la creación de una interfaz gráfica interactiva y la publicación del sistema en plataformas accesibles mediante un navegador web. Las principales tecnologías utilizadas fueron SWI-Prolog, HTML, CSS, JavaScript, Docker, Git, GitHub, Render y Vercel. Cada una desempeña una función específica dentro de la arquitectura del sistema.

-SWI-Prolog

SWI-Prolog constituye la tecnología principal del proyecto, debido a que permite implementar el motor de inferencia y representar la base de conocimientos mediante hechos y reglas lógicas.

En SEOV-Pro se utilizó para registrar las carreras profesionales, establecer relaciones con sus habilidades y entornos laborales, y desarrollar reglas capaces de determinar diferentes niveles de compatibilidad con el perfil seleccionado por el estudiante. Además, SWI-Prolog proporciona bibliotecas para implementar servidores HTTP, lo que permitió recibir solicitudes desde la interfaz gráfica y devolver recomendaciones en formato JSON.

-HTML

En SEOV-Pro se empleó para construir los elementos principales de la interfaz gráfica, incluyendo encabezados, formularios, etiquetas, campos de selección, botones y contenedores destinados a mostrar las recomendaciones. La utilización de HTML permitió organizar la información de manera estructurada y establecer los componentes con los que interactúa el estudiante.

-CSS

En el proyecto se empleó para establecer los colores, tamaños, espacios, tipografías y estilos de los diferentes componentes de la interfaz. SEOV-Pro utiliza una presentación visual de tonalidades oscuras, con elementos diferenciados para facilitar la identificación del formulario, los controles de interacción y los resultados obtenidos.

-JavaScript

JavaScript es un lenguaje de programación utilizado para incorporar comportamiento dinámico e interactividad en aplicaciones web. En SEOV-Pro se utilizó para obtener los valores seleccionados en el formulario, construir las solicitudes dirigidas al servidor SWI-Prolog, procesar las respuestas recibidas y actualizar dinámicamente los resultados mostrados en la página.

Una de las funciones principales utilizadas es fetch(), que permite realizar solicitudes HTTP desde el navegador hacia el backend. JavaScript también interviene en las funciones de consulta y limpieza del formulario, facilitando que el estudiante realice diferentes evaluaciones sin necesidad de interactuar directamente con el código Prolog.

-Docker

En SEOV-Pro se utilizó para preparar el backend desarrollado en SWI-Prolog y facilitar su ejecución en la plataforma Render. Mediante un archivo Dockerfile se establecen las instrucciones necesarias para construir la imagen del contenedor, instalar el entorno requerido y ejecutar el servidor del sistema experto.

-Git

Git es un sistema de control de versiones distribuido que permite registrar y administrar los cambios realizados en los archivos de un proyecto. Durante el desarrollo de SEOV-Pro se utilizó para mantener un historial de modificaciones y organizar las actualizaciones realizadas al código fuente.

Mediante operaciones como add, commit y push, fue posible preparar los cambios, registrar versiones y enviarlas al repositorio remoto. Esta herramienta contribuyó a mantener un proceso de desarrollo ordenado y a facilitar el seguimiento de las modificaciones efectuadas.

-GitHub

GitHub es una plataforma de alojamiento de repositorios Git que permite almacenar, compartir y administrar proyectos de software.

En SEOV-Pro se utilizó para publicar el código fuente del frontend y backend, así como los archivos necesarios para la contenerización y la documentación del sistema. El repositorio contiene los archivos principales del proyecto, incluyendo index.html, main.pl, servidor_web.pl, Dockerfile y README.md. Además, GitHub permitió vincular el proyecto con las plataformas utilizadas para el despliegue de la aplicación.

El repositorio público se encuentra disponible en:

https://github.com/Monse-Teran/SEOV-Pro

-Render

Render es una plataforma de alojamiento en la nube que permite desplegar aplicaciones y servicios web.

En SEOV-Pro se utilizó para alojar el backend desarrollado en SWI-Prolog mediante un contenedor Docker. El servicio recibe las solicitudes HTTP provenientes de la interfaz web, ejecuta las consultas correspondientes en el motor de inferencia y devuelve los resultados en formato JSON.

La dirección pública del backend es:

https://seov-pro-backend.onrender.com

La utilización de Render permitió publicar el motor lógico sin necesidad de mantener un servidor físico propio. Debido a que se utiliza un servicio gratuito, pueden presentarse tiempos de espera adicionales cuando el backend permanece inactivo y debe volver a iniciar.

-Vercel

Vercel es una plataforma orientada al despliegue y alojamiento de aplicaciones web.

En SEOV-Pro se utilizó para publicar el frontend desarrollado con HTML, CSS y JavaScript, permitiendo que los estudiantes accedan a la interfaz gráfica mediante un navegador.

La aplicación web se encuentra disponible en:

https://seov-pro-frontend.vercel.app

La separación del frontend y el backend permite que ambos componentes se desplieguen y mantengan de manera independiente.

-HTTP y JSON

Además de los lenguajes y plataformas mencionados, SEOV-Pro utiliza tecnologías estándar para establecer la comunicación entre sus componentes. HTTP permite enviar solicitudes desde el navegador hacia el servidor SWI-Prolog. Por su parte, JSON se utiliza para representar los resultados en un formato estructurado que puede ser interpretado fácilmente por JavaScript. Esta combinación permite intercambiar información entre el frontend y el backend, independientemente de los lenguajes de programación utilizados en cada componente.

-Integración de las tecnologías

La integración de las herramientas utilizadas permitió desarrollar una aplicación web funcional basada en programación lógica.

HTML y CSS proporcionan la estructura y presentación visual; JavaScript gestiona la interacción con el usuario y la comunicación HTTP; SWI-Prolog ejecuta las reglas de inferencia; Docker proporciona el entorno de ejecución del backend; Render y Vercel permiten publicar los componentes en internet; y Git junto con GitHub facilitan la administración del código fuente.

Procedimiento de integración y despliegue

-Descripción general del procedimiento

El proceso de integración y despliegue del Sistema Experto de Orientación Vocacional (SEOV-Pro) consistió en organizar los componentes del proyecto, configurar el servidor desarrollado en SWI-Prolog, establecer la comunicación con la interfaz gráfica y publicar ambos componentes en plataformas de alojamiento web.

Para realizar este procedimiento se utilizó una arquitectura cliente-servidor, en la cual el frontend y el backend se encuentran alojados de manera independiente. El frontend, desarrollado mediante HTML, CSS y JavaScript, fue publicado en Vercel. Por su parte, el backend, implementado mediante SWI-Prolog, fue desplegado en Render utilizando un contenedor Docker. GitHub se utilizó como repositorio central para almacenar el código fuente, mantener el historial de modificaciones y facilitar la integración con los servicios de despliegue.

-Organización de los archivos del proyecto

Como primera etapa, se organizó el código fuente en carpetas independientes para separar las responsabilidades del frontend y el backend.

La carpeta backend contiene los archivos relacionados con la programación lógica, el servidor HTTP y la configuración del contenedor Docker.

Dentro de esta carpeta se encuentran los siguientes archivos:

- main.pl: contiene el punto de entrada de la aplicación y las instrucciones necesarias para iniciar el servidor SWI-Prolog.
- servidor_web.pl: contiene la implementación del servidor HTTP, la base de conocimientos, las reglas de inferencia y los procedimientos necesarios para atender las consultas.
- Dockerfile: define las instrucciones para construir el contenedor utilizado durante el despliegue del backend.

Por otra parte, la carpeta frontend contiene el archivo index.html, en el cual se implementa la interfaz gráfica y la lógica JavaScript encargada de comunicarse con el servidor. Adicionalmente, se incorporaron archivos de documentación y configuración, como README.md y .gitignore.

-Configuración del servidor SWI-Prolog

Una vez organizada la estructura del proyecto, se configuró el servidor HTTP desarrollado mediante SWI-Prolog.

El archivo main.pl se utiliza como punto de entrada del backend. Su función consiste en cargar el archivo que contiene el servidor web, identificar el puerto de ejecución e iniciar el servicio. Para facilitar el despliegue en Render, el servidor fue configurado para utilizar el puerto indicado mediante la variable de entorno PORT. En caso de que esta variable no esté definida, se utiliza el puerto 8080 como valor predeterminado.

-Integración del frontend con el backend

La integración entre los componentes se realizó mediante solicitudes HTTP enviadas desde JavaScript hacia el servidor SWI-Prolog. Para ello, se utilizó la función fetch(), que permite enviar los parámetros del formulario y recibir respuestas estructuradas en formato JSON.

Cuando el estudiante selecciona el área de interés, la habilidad y el entorno laboral, JavaScript construye una dirección de consulta que contiene estos valores. El servidor recibe la solicitud, ejecuta las reglas de inferencia correspondientes y devuelve las carreras identificadas junto con sus niveles de compatibilidad.

Posteriormente, JavaScript procesa la respuesta y actualiza la sección de resultados de la interfaz gráfica. Debido a que el frontend y el backend se encuentran alojados en dominios diferentes, también fue necesario contemplar la configuración de CORS, mecanismo que permite controlar las solicitudes realizadas entre distintos orígenes web.

-Creación del repositorio en GitHub

Para almacenar y administrar el código fuente del proyecto se creó un repositorio en GitHub con el nombre SEOV-Pro.

El repositorio permitió centralizar los archivos del frontend y backend, registrar las modificaciones realizadas durante el desarrollo y mantener una versión accesible del sistema. Se utilizó Git como herramienta de control de versiones para preparar los archivos, generar commits y enviar los cambios al repositorio remoto.

-Configuración de la comunicación entre servicios

Una vez publicados ambos componentes, se configuró la interfaz gráfica para que las solicitudes realizadas mediante JavaScript fueran dirigidas a la dirección pública del backend alojado en Render.

De esta manera, la aplicación dejó de depender de una dirección local y comenzó a utilizar un servicio accesible a través de internet.

El flujo de comunicación quedó establecido de la siguiente manera:

1. El usuario accede al frontend publicado en Vercel.
2. Selecciona las características de su perfil vocacional.
3. JavaScript envía una solicitud HTTP al backend de Render.
4. SWI-Prolog procesa la consulta utilizando su base de conocimientos.
5. El servidor devuelve las recomendaciones en formato JSON.
6. La interfaz presenta los resultados al estudiante.

Esta integración permitió comprobar el funcionamiento conjunto de los componentes y validar la arquitectura cliente-servidor propuesta.

-Resultado del procedimiento de despliegue

El procedimiento de integración y despliegue permitió publicar una versión funcional de SEOV-Pro, accesible mediante internet y compuesta por una interfaz gráfica y un motor de inferencia ejecutados en plataformas independientes. La utilización de GitHub, Docker, Render y Vercel facilitó la organización, publicación y mantenimiento del proyecto.

Problemas encontrados y soluciones implementadas

-Reconocimiento del comando SWI-Prolog en Windows

Durante las pruebas iniciales del backend se presentó una dificultad al intentar ejecutar SWI-Prolog desde la terminal de Windows mediante el comando swipl. El sistema no reconocía el comando, lo que impedía iniciar directamente el intérprete desde la consola.

Causa identificada:

El ejecutable de SWI-Prolog no se encontraba disponible mediante la variable de entorno PATH, utilizada por Windows para localizar programas ejecutables.

Solución aplicada:

Se identificó la ubicación del archivo ejecutable de SWI-Prolog y se utilizó su ruta completa para iniciar el programa desde PowerShell. De esta manera, fue posible ejecutar el intérprete sin depender de que el comando estuviera registrado en las variables de entorno del sistema.

-Bloqueo de una biblioteca de SWI-Prolog por las políticas de seguridad de Windows

Después de ejecutar SWI-Prolog mediante su ruta completa, el programa mostraba un mensaje indicando que el servidor había iniciado en el puerto configurado.

Sin embargo, al intentar acceder al servicio desde un navegador, la conexión no se completaba correctamente y se presentaba el error ERR_EMPTY_RESPONSE. Inicialmente, este comportamiento podía interpretarse como un problema de configuración del servidor o de comunicación HTTP.

Causa identificada:

Mediante la revisión del Visor de eventos de Windows se detectó que una política de integridad de código impedía cargar la biblioteca http_stream.dll, utilizada por SWI-Prolog para determinadas operaciones de comunicación HTTP.

El registro del sistema, correspondiente al evento 3077 de integridad de código, permitió identificar que el bloqueo estaba relacionado con restricciones de seguridad del entorno Windows.

Solución aplicada:

Se determinó que el problema correspondía al entorno local y no necesariamente al código lógico del sistema experto. En lugar de modificar las políticas de seguridad del equipo, se continuó con el despliegue del backend mediante un contenedor Docker en Render.

Este entorno permitió ejecutar SWI-Prolog de forma independiente de las restricciones específicas del equipo Windows. Posteriormente, se comprobó que el backend publicado respondía a las solicitudes HTTP y generaba recomendaciones en formato JSON. Por lo tanto, aunque la restricción local no fue eliminada, se implementó una alternativa de ejecución que permitió continuar con el proyecto y disponer de una versión funcional en internet.

-Integración entre el frontend y el backend

Una de las principales dificultades del proyecto consistió en integrar una interfaz desarrollada con HTML, CSS y JavaScript con un motor de inferencia implementado en SWI-Prolog. Ambos componentes utilizan tecnologías diferentes y requieren un mecanismo de comunicación que permita enviar las preferencias del estudiante y recuperar las recomendaciones generadas.

Causa identificada:

La interfaz gráfica no puede ejecutar directamente las reglas de SWI-Prolog dentro del navegador, por lo que era necesario implementar un servicio que recibiera solicitudes y devolviera los resultados en un formato compatible con JavaScript.

Solución aplicada:

Se implementó un servidor HTTP en SWI-Prolog y se estableció un endpoint denominado /recomendar. Mediante JavaScript y la función fetch(), el frontend envía los parámetros correspondientes al área de interés, la habilidad destacada y el entorno laboral.

El backend procesa estos valores mediante las reglas de inferencia y devuelve las recomendaciones en formato JSON. También se contempló la configuración de CORS para permitir la comunicación entre los dominios utilizados por Vercel y Render. Esta integración permitió establecer un funcionamiento conjunto entre la interfaz gráfica y el motor lógico.

Conclusiones

Lopez Teran Monserrat

El desarrollo del proyecto SEOV-Pro (Sistema Experto de Orientación Vocacional) representó una experiencia de aprendizaje significativa, ya que me permitió comprender y aplicar los fundamentos de la programación lógica mediante el lenguaje SWI-Prolog, así como conocer su utilidad en la construcción de sistemas expertos capaces de generar recomendaciones a partir de conocimientos previamente establecidos.

Durante la realización del proyecto, uno de los principales aprendizajes que fue comprender la diferencia entre la programación tradicional y la programación declarativa. A través de Prolog pude observar cómo es posible resolver problemas mediante la definición de hechos, relaciones y reglas de inferencia, en lugar de depender exclusivamente de instrucciones secuenciales.

El proyecto me permitió ampliar mis conocimientos sobre el desarrollo de aplicaciones web, debido a que fue necesario integrar el motor de inferencia desarrollado en SWI-Prolog con una interfaz gráfica construida mediante HTML, CSS y JavaScript. Esta integración me ayudó a comprender el funcionamiento de la arquitectura cliente-servidor, el intercambio de información mediante solicitudes HTTP y la utilización del formato JSON para comunicar aplicaciones desarrolladas con diferentes tecnologías.

También aprendí a utilizar herramientas como Git y GitHub para organizar y administrar el código fuente, así como Vercel y Render para desplegar los componentes de la aplicación. Considero que estos conocimientos son especialmente valiosos porque me permitieron comprender que el desarrollo de software no termina cuando el programa funciona localmente, sino que también implica prepararlo, configurarlo y ponerlo a disposición de los usuarios. Finalmente, considero que SEOV-Pro cumplió con su propósito de demostrar cómo la programación lógica puede aplicarse en un problema relacionado con la orientación vocacional. Aunque el sistema puede ampliarse mediante la incorporación de nuevas carreras, habilidades y criterios de evaluación, su desarrollo me permitió fortalecer mis capacidades de análisis, razonamiento lógico, investigación y resolución de problemas.

En conclusión, este proyecto me brindó una visión más completa del proceso de desarrollo de un sistema experto, desde la construcción de su base de conocimientos hasta su integración y despliegue como aplicación web.

Garcia Ramirez Saul Maximiliano

En conclusión, el avance del proyecto SEOV-Pro (Sistema Experto de Orientación Vocacional) representa un esfuerzo continuo por aplicar los fundamentos de la programación lógica en la resolución de problemas prácticos, como el apoyo en la toma de decisiones profesionales. Al ser un proyecto en desarrollo, su estado actual refleja una evolución constante hacia la construcción de una herramienta robusta y escalable.

Hasta esta fase, la implementación del motor de inferencia en SWI-Prolog ha permitido estructurar de manera efectiva una base de conocimientos, demostrando la viabilidad de utilizar un enfoque declarativo basado en hechos y reglas lógicas. De igual forma, el progreso del proyecto ha requerido la construcción de una arquitectura cliente-servidor funcional, logrando integrar este motor lógico con una interfaz web (desarrollada con HTML, CSS y JavaScript) mediante solicitudes HTTP y el intercambio de datos en formato JSON.

El ciclo iterativo de este trabajo se ha respaldado mediante el uso de herramientas de control de versiones como Git y GitHub, junto con entornos de despliegue en la nube como Vercel y Render. Esto ha sido fundamental para gestionar el código fuente y mantener los componentes accesibles para pruebas y futuras actualizaciones, comprendiendo que la puesta en producción es un proceso activo y no solo un paso final.

Dado que el sistema sigue en desarrollo, se mantiene la perspectiva de ampliar progresivamente su alcance mediante la incorporación de nuevas carreras, habilidades y criterios de evaluación más complejos. En suma, el trabajo realizado hasta el momento ha proporcionado una visión integral del ciclo de creación de software, fortaleciendo de manera significativa las capacidades de análisis, abstracción lógica y resolución de problemas técnicos frente a los retos que supone la integración de tecnologías dispares.

Referencias bibliográficas

- Docker, Inc. (s. f.). *What is Docker?* Docker Docs. https://docs.docker.com/get-started/docker-overview/

- GitHub, Inc. (s. f.). *GitHub Docs*. https://docs.github.com/

- Mozilla. (s. f.). *Fetch API*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

- Mozilla. (s. f.). *HTML: HyperText Markup Language*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/HTML

- Mozilla. (s. f.). *CSS: Cascading Style Sheets*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/CSS

- Mozilla. (s. f.). *JavaScript*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/JavaScript

- Render. (s. f.). *Docker on Render*. Render Docs. https://render.com/docs/docker
- Render. (s. f.). *Your first Render deploy*. Render Docs. https://render.com/docs/your-first-deploy

- SWI-Prolog. (s. f.). *SWI-Prolog reference manual*. https://www.swi-prolog.org/pldoc/doc_for?object=manual

- Vercel. (s. f.). *Deploying to Vercel*. Vercel Docs. https://vercel.com/docs/deployments

Declaración del uso de herramientas de inteligencia artificial

Durante el desarrollo del proyecto SEOV-Pro (Sistema Experto de Orientación Vocacional) se utilizó la herramienta de inteligencia artificial generativa ChatGPT, desarrollada por OpenAI, como recurso de apoyo para la comprensión de conceptos técnicos y la resolución de dudas.

1. **Integración de tecnologías:** asistencia para comprender la comunicación entre SWI-Prolog y JavaScript mediante solicitudes HTTP y respuestas en formato JSON.
2. **Configuración y despliegue:** orientación durante la organización del repositorio en GitHub, la preparación del backend mediante Docker y la publicación de los componentes del proyecto en Render y Vercel.
3. **Identificación de problemas técnicos:** apoyo para interpretar mensajes de error, analizar posibles causas y proponer alternativas de solución durante las etapas de ejecución e integración del sistema.